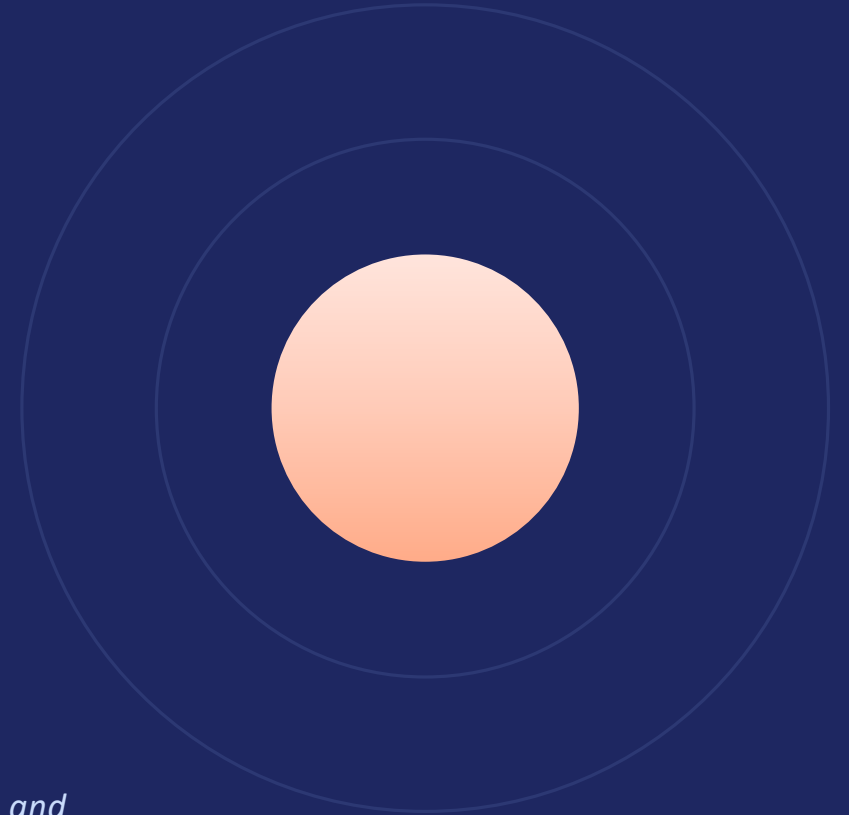


BUSINESS IMPACT AGENT

# Self-service impact analysis for rule changes



*A working prototype that answers two questions before a ticket is raised: is this change feasible, and who else needs to know.*

**Nima Behravan**<behravan.nima@gmail.com>

Hackathon prototype · Large airline digital unit · 2026

# The problem

A business rule rarely lives in one place. It's written up somewhere, implemented somewhere else, consumed by several services, and referenced in tickets nobody has cross-checked.

1

## No shared view

Documentation, tickets, service ownership, and code live in four separate systems with no single place to check impact.

2

## Manual, informal cross-checks

Validating a change means searching multiple tools by hand, then asking around to find out who is affected.

3

## Downstream teams find out too late

A change often reaches the consuming team only after a ticket is already in development, when the fix costs far more.

“

*A wait-time rule changed for one customer segment and reached the front-end team only after the ticket was already in development.*

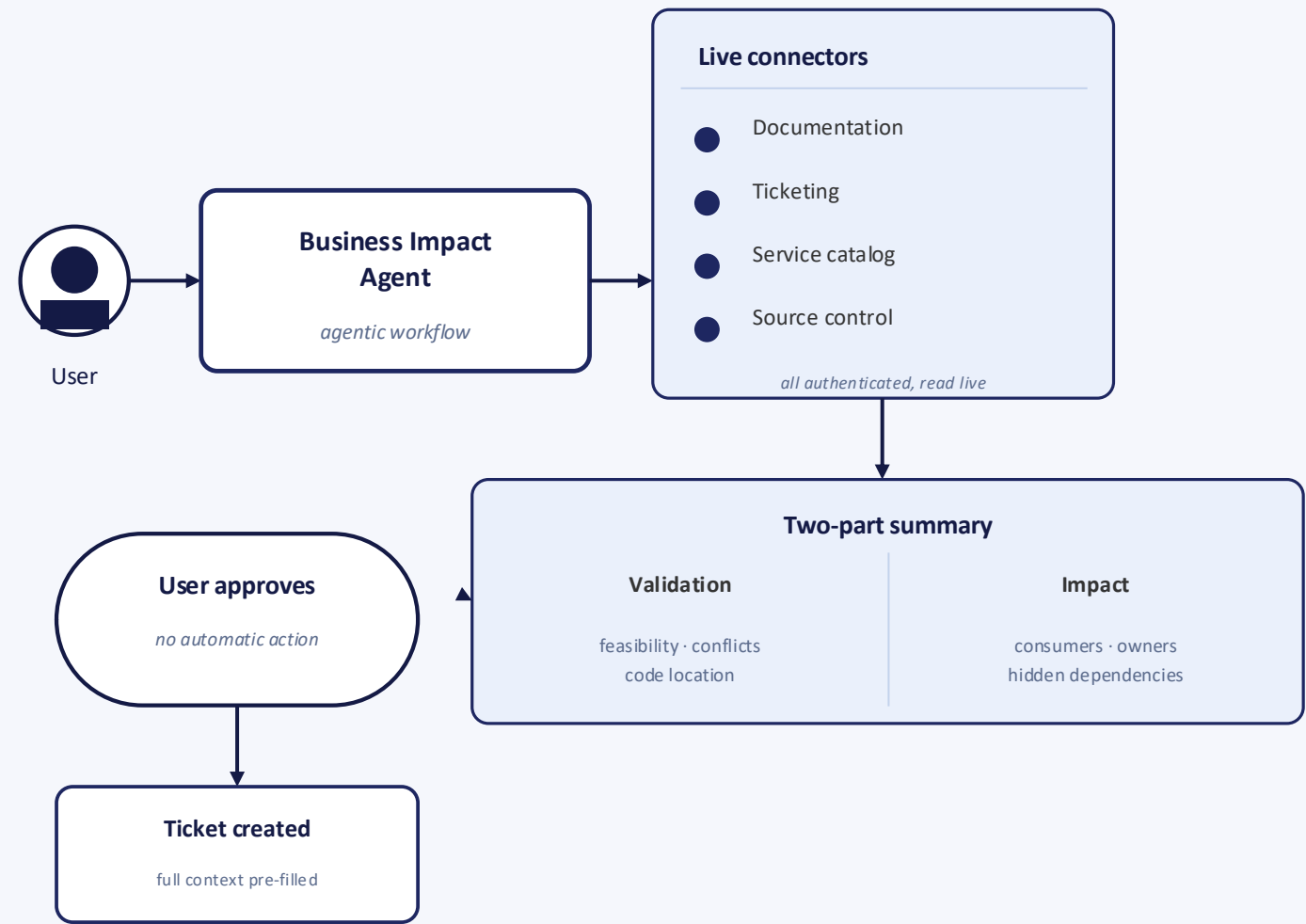
— the real-world trigger for this build

4

systems to check by hand, every time

# How it works

Built and demonstrated end to end



## Future direction

### Automatic change detection

Change descriptions generated from document diffs and schema changes

### Persistent backend

Full audit trail and reusable impact lookups

### Unregistered dependencies

Surfaced from code and data lineage, not just the service catalog

### Severity-based routing

Notify, ticket, or log only, based on impact

# Every request gets the same two answers



*Kept separate on purpose, so the reasoning stays auditable rather than a single black-box verdict.*



## Validation

*Can this change be made?*

---

### Current state

What the rule looks like today, pulled live from documentation.

---

### Feasibility & conflicts

Whether the change is structurally straightforward, and any open work that overlaps.

---

### Code location

Exactly where the implementation lives, so the change isn't a search exercise.



## Impact

*Who else needs to know?*

---

### Affected components

Every downstream service or component the rule feeds into.

---

### Owning teams

Who to notify, pulled from the service catalog rather than tribal knowledge.

---

### Hidden dependencies

Consumers a few steps removed that a manual check would likely miss.

# Demonstrated on a real change



A regulatory update unified a customer-eligibility wait-time threshold to a single value across several previously inconsistent configurations. Given only a plain-language description, the agent ran the full workflow unattended.

4

**systems queried in parallel**

---

*documentation, tickets, service  
catalog, source control*

3

**affected components found**

---

*spanning two separate owning teams*

2

**hidden dependencies  
surfaced**

---

*not obvious from the rule definition  
itself*

72h

**unified threshold identified**

---

*with per-configuration current values  
pulled live*

*One connector hit an authentication error mid-run. Rather than skip it silently, the agent flagged it for a manual check, an explicit trade-off from keeping a human in the loop.*

# What makes this different



## Lineage & process-mining tools

- Analytical and passive — describe what's connected after the fact
- You have to go looking for the answer
- Reconstruct history from logs and metadata
- Valuable for audit and discovery, not for stopping a problem before it starts

## Business Impact Agent

- Active and triggered at the moment someone proposes a change
- Takes a plain-language description as input
- Produces a validated, cross-team answer before any ticket exists
- Ends in an action: an approved, fully-documented ticket